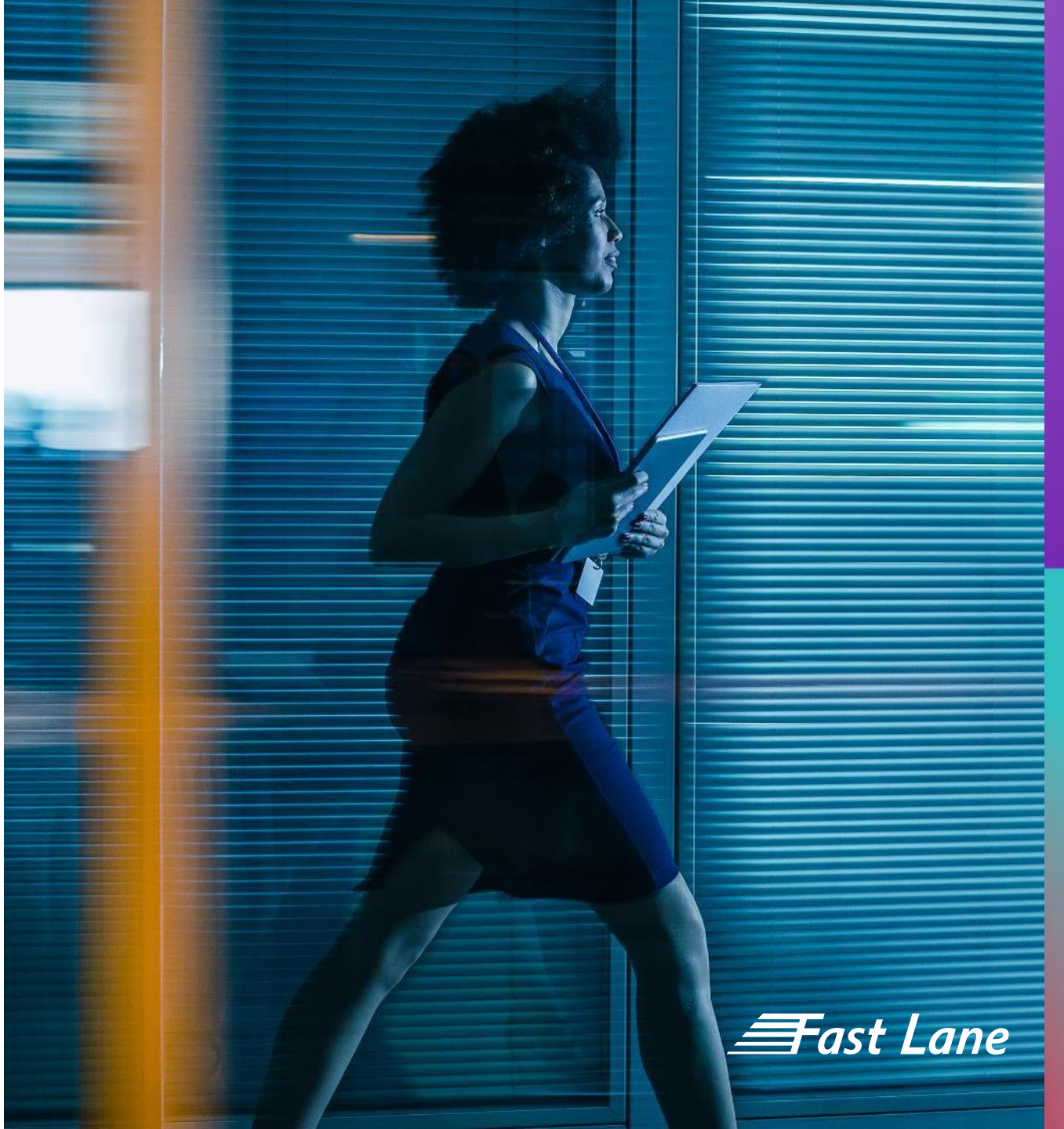
Fast Lane Group

Worldwide Education & Professional Services

Microsoft Solutions Partner

Microsoft Cloud Training Services



www.flane.de | © 2026 Fast Lane Institute for Knowledge Transfer GmbH

— AGENTIC AI: FROM STRATEGY TO **PRODUCTION.**

Alexander Kastil

Full stack, from enterprise software to agentic AI. Since 1996

AI at Work
Value discovery, KPI mapping and roadmap

Enterprise AI
Governed multi-agent systems on Microsoft Copilot and Foundry

Agentic Software Engineering
Harness architecture for Claude Code and GitHub Copilot

Team Enablement
AI driven content production and delivery

integrations.at
alexander.kastil@integrations.at

github.com/alexander-kastil
linkedin.com/in/alexander-kastil

INTEGRATIONS

AGENTIC AI CONSULTING · VIENNA



Microsoft CERTIFIED — AGENTIC AI BUSINESS SOLUTIONS ARCHITECT — ★★★

Microsoft CERTIFIED — AZURE AI APP AND AGENT DEVELOPER — ASSOCIATE ★★

Microsoft CERTIFIED — DEVOPS ENGINEER — EXPERT ★★★

SCAN TO CONNECT
integrations.at

MODULE 3

Hosting on Container Apps & AKS

Putting the Contoso Logistics services on Azure

Event-Driven Distributed Apps on Azure: Aspire, Dapr & Durable Execution



01

Aspire Deploy: ACR & Container Apps

The registry, the environment and the apps from the AppHost

02

Key Vault & App Configuration

Managed identity, secrets, settings and feature flags

03

Revisions: Traffic Splitting & Draining

Revision modes, traffic weights and graceful shutdown

04

AKS: Aspire-Generated Kubernetes Manifests

Helm charts, the AKS environment and the Dapr extension

05

Application Insights: Traces & KQL

The Azure Monitor distro, trace context and KQL

Hosting on Container Apps and AKS

Putting the Contoso Logistics services on Azure

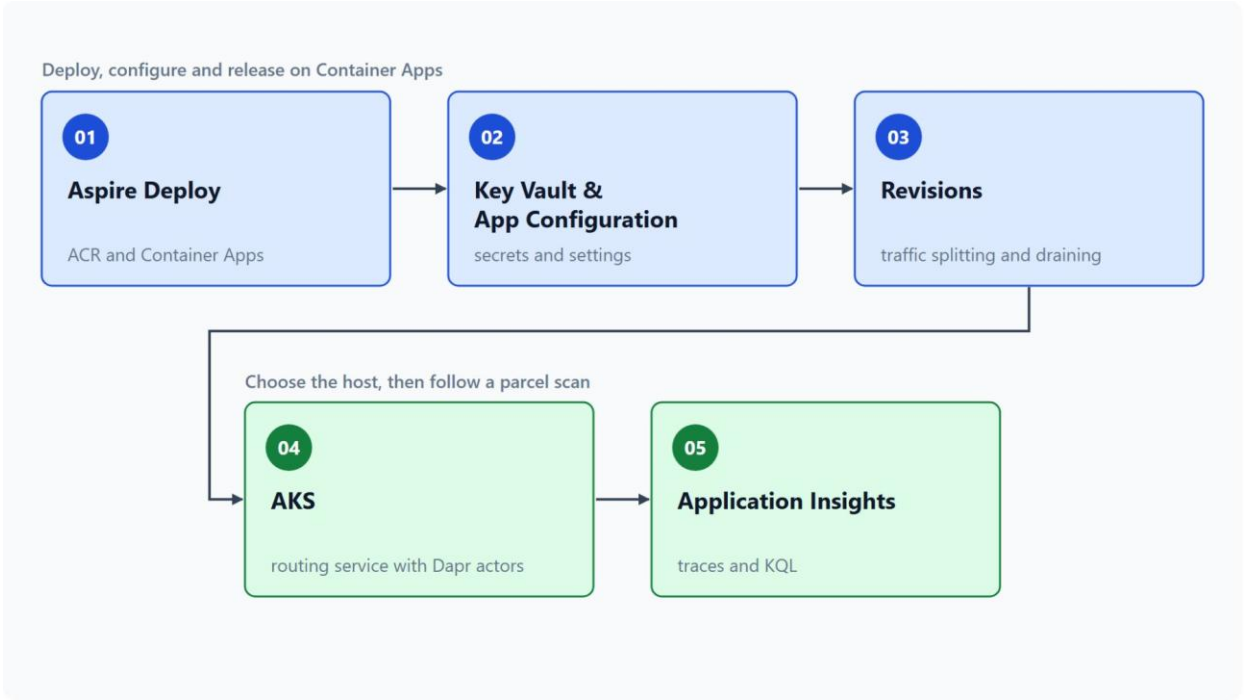
- **Deploy**: `aspire deploy` puts the AppHost on Container Apps, the routing service runs on AKS
- **Configure**: secrets from Key Vault and settings from App Configuration over a managed identity
- **Release and trace**: revisions shift traffic, Application Insights follows a parcel scan



Overview

The path through the five topics

- Deploy the AppHost to Container Apps with **aspire deploy**
- Keep secrets in **Key Vault** and settings in **App Configuration**
- Release a new version behind a **traffic split** and drain consumers
- Host the routing service on **AKS** with the Dapr extension
- Follow a parcel scan in **Application Insights** with KQL



01

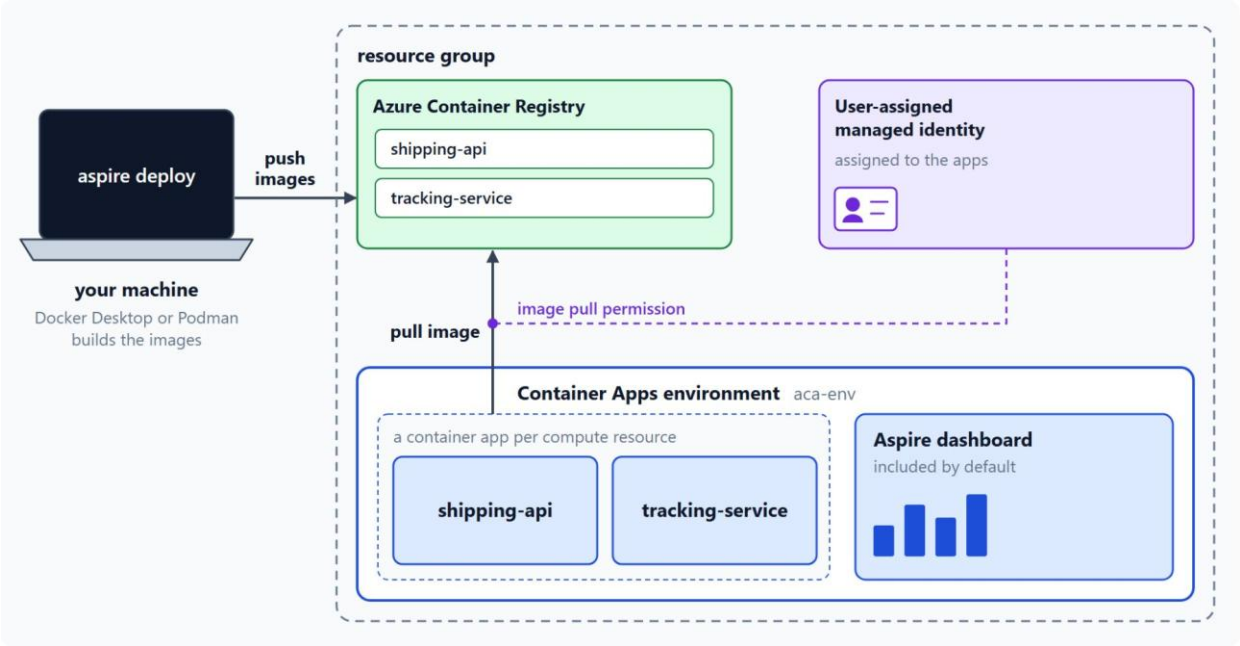
Aspire Deploy: ACR & Container Apps

The registry, the environment and the apps from the AppHost

Deploying with aspire deploy

The AppHost declares the environment, one command provisions the rest

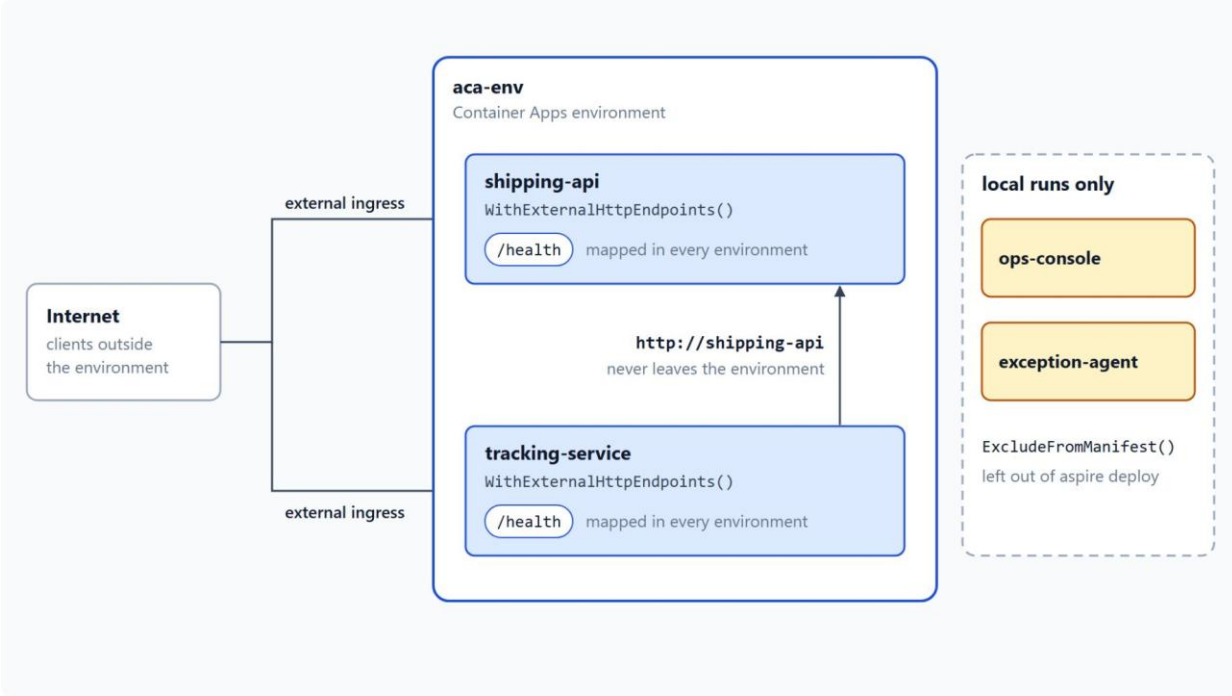
- `AddAzureContainerAppEnvironment` declares the environment in the AppHost
- `aspire deploy` provisions, builds and pushes the images, then deploys the apps
- The **managed identity** pulls the images, so you pass no registry credentials
- A CI job can build in the registry with `az acr build`
- `aspire destroy --yes` deletes the whole resource group



Ingress and calls by app name

Public endpoints only where outside clients need them

- `WithExternalHttpEndpoints` publishes an endpoint; without it a service stays internal
- `shipping-api` and `tracking-service` are the two public services
- Apps in the environment call each other by name: `http://shipping-api`
- `/health` maps in every environment, so it can be probed from outside
- `ExcludeFromManifest()` keeps `ops-console` and `exception-agent` local



02

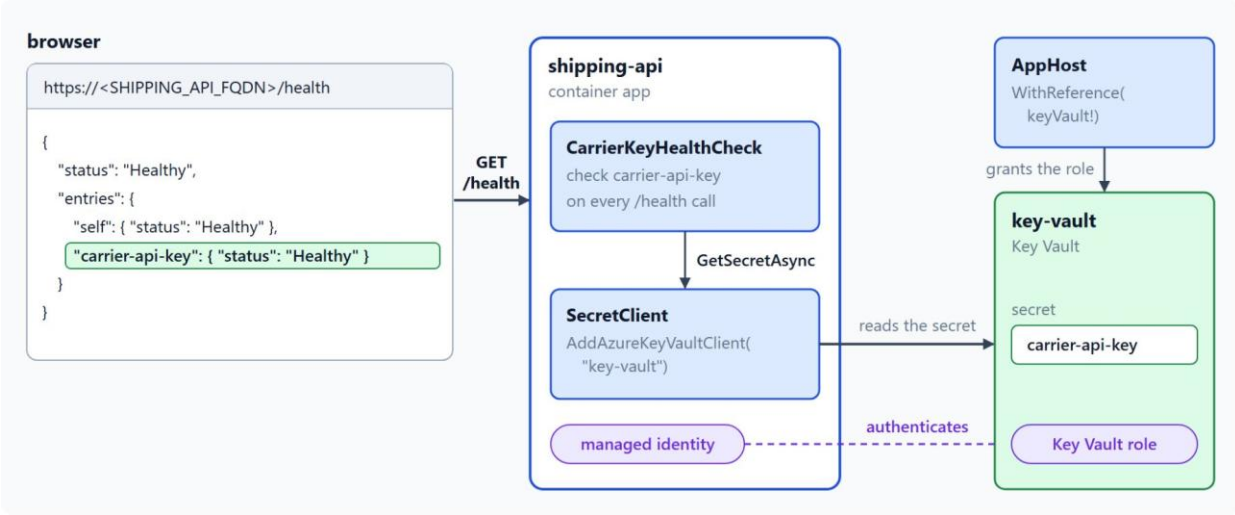
Key Vault & App Configuration

Managed identity, secrets, settings and feature flags

Key Vault access with a managed identity

The same code runs on a laptop and in Azure

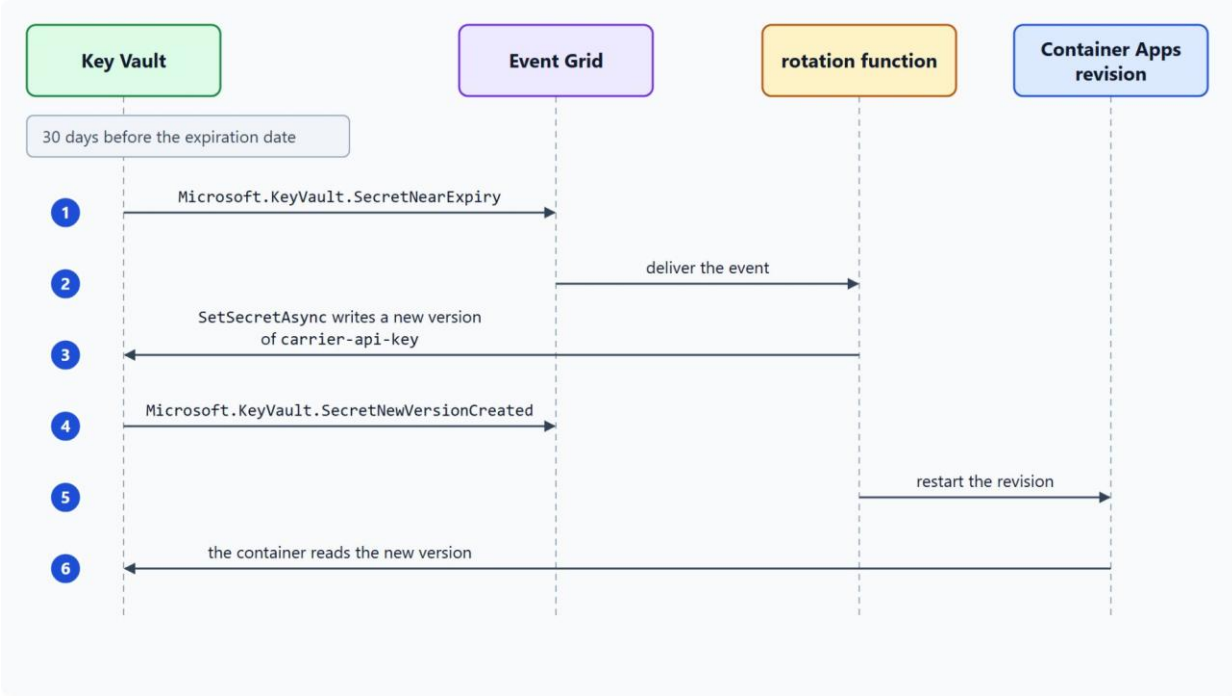
- `DefaultAzureCredential` uses the Azure CLI sign-in locally and the **managed identity** in Azure
- `SecretClient` reads secrets with that credential, so no password is stored
- Read access comes from the **Key Vault Secrets User** role, scoped to the vault
- `AddAzureKeyVault` and `WithReference` let Aspire assign the role
- The default is Key Vault Administrator; `WithRoleAssignments` narrows it



Secret versions and rotation

Key Vault events drive the rotation

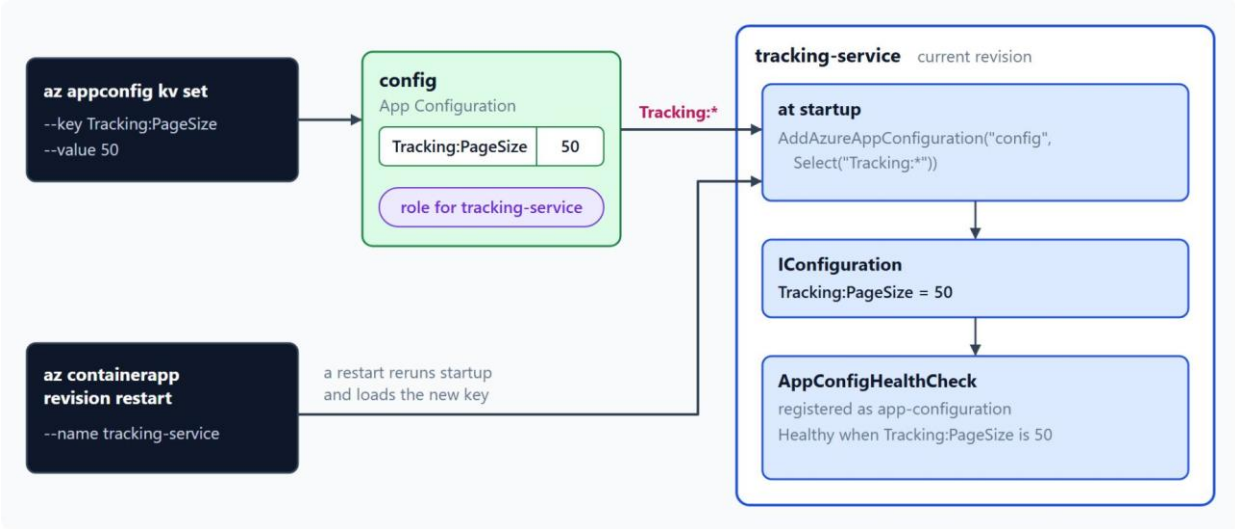
- Each write creates a new **secret version**: `SetSecretAsync`
- Key Vault raises `SecretNearExpiry` 30 days before the expiration date
- Event Grid delivers it to a rotation function that writes the new version
- Restart the Container Apps revision so the container reads it
- Prefer managed identity; rotate only where a key is still needed



App Configuration and feature flags

Settings load at startup, flags refresh on their own

- The .NET provider loads key-values and feature flags into `IConfiguration`
- A **Key Vault reference** points to a secret that stays in the vault
- The AppHost declares the store in publish mode, `WithReference` grants a role
- Keys load at startup, so a new key needs a **revision restart**
- `UseFeatureFlags` and `IFeatureManager`, refreshed every 30 seconds



03

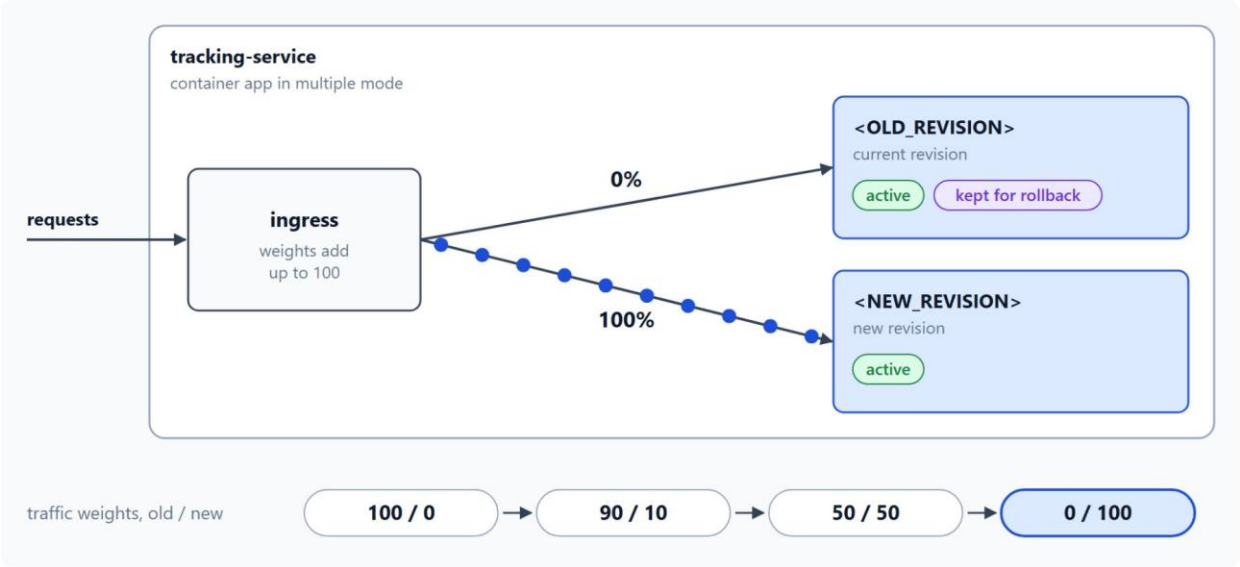
Revisions: Traffic Splitting & Draining

Revision modes, traffic weights and graceful shutdown

Revision modes and traffic splitting

Weighted traffic per revision and rollback

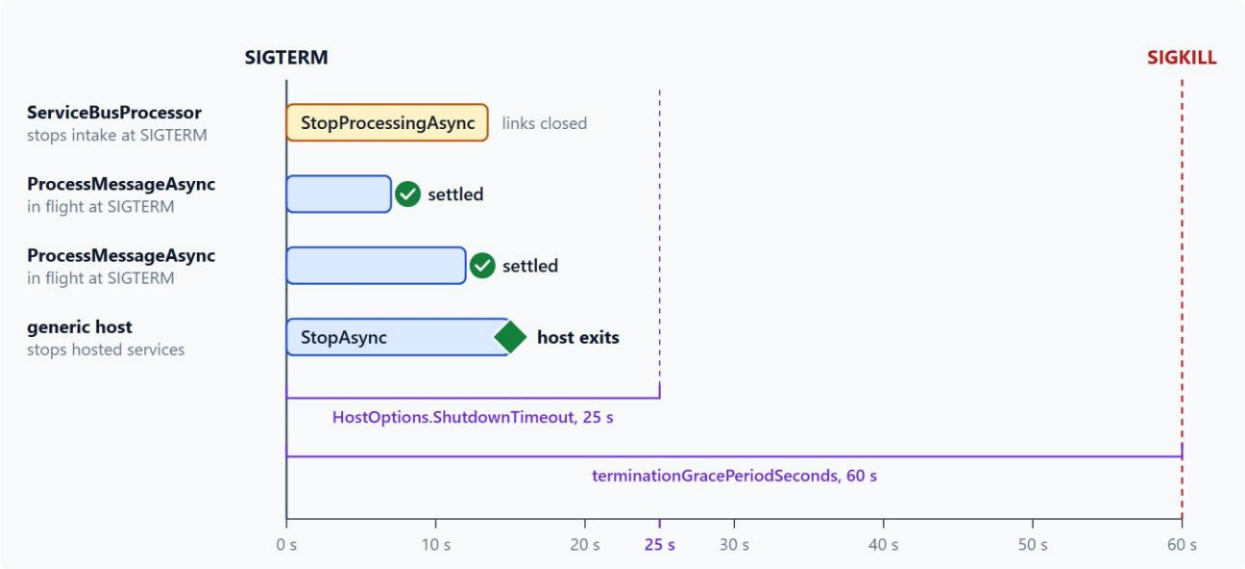
- **Single** mode is the default: the new revision takes over once ready
- **Multiple** mode keeps several revisions active for blue-green and canary
- `az containerapp ingress traffic set` sets weights that add up to 100
- Roll back with the same command and the previous revision at 100
- **Labels** (preview) give a revision a stable URL



Graceful shutdown for message consumers

In-flight messages settle before SIGKILL

- A change to the **template** creates a revision; app-wide settings do not
- SIGTERM, then `terminationGracePeriodSeconds` (default 30 s), then SIGKILL
- Set the grace period longer than `HostOptions.ShutdownTimeout`
- `StopProcessingAsync` stops intake and waits for in-flight handlers
- Settle each message before the handler returns



04

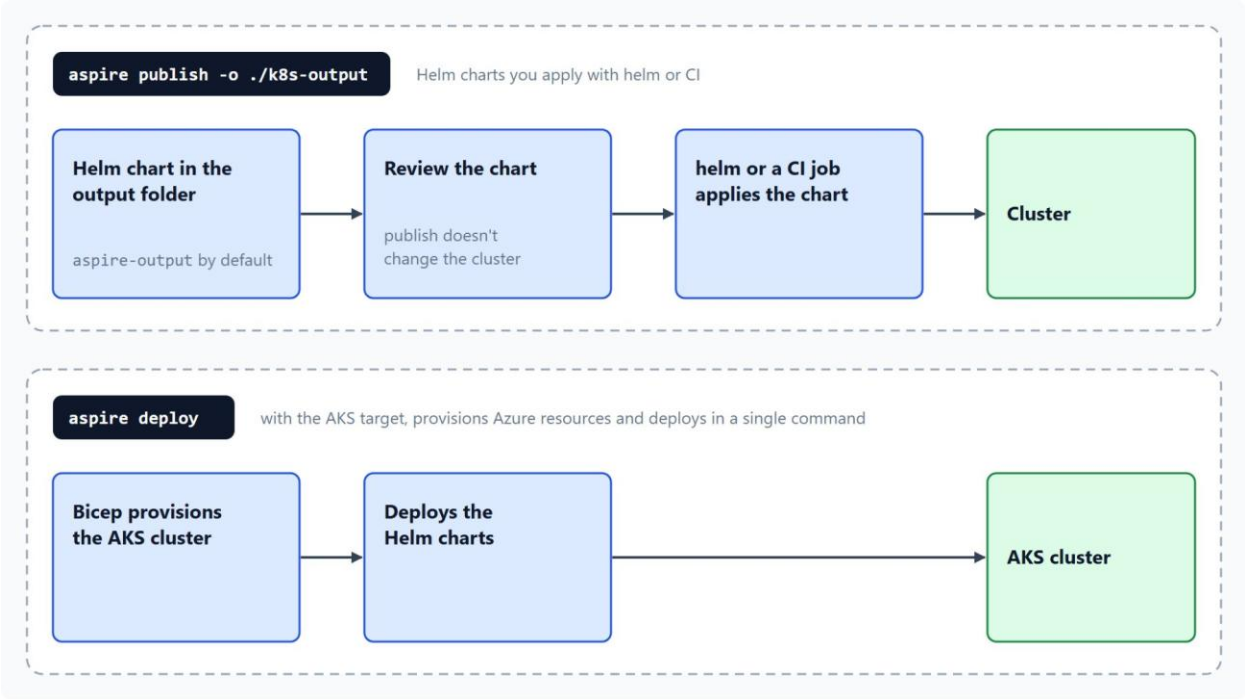
AKS: Aspire-Generated Kubernetes Manifests

Helm charts, the AKS environment and the Dapr extension

Helm charts and the AKS target

Publish for a pipeline or deploy in one step

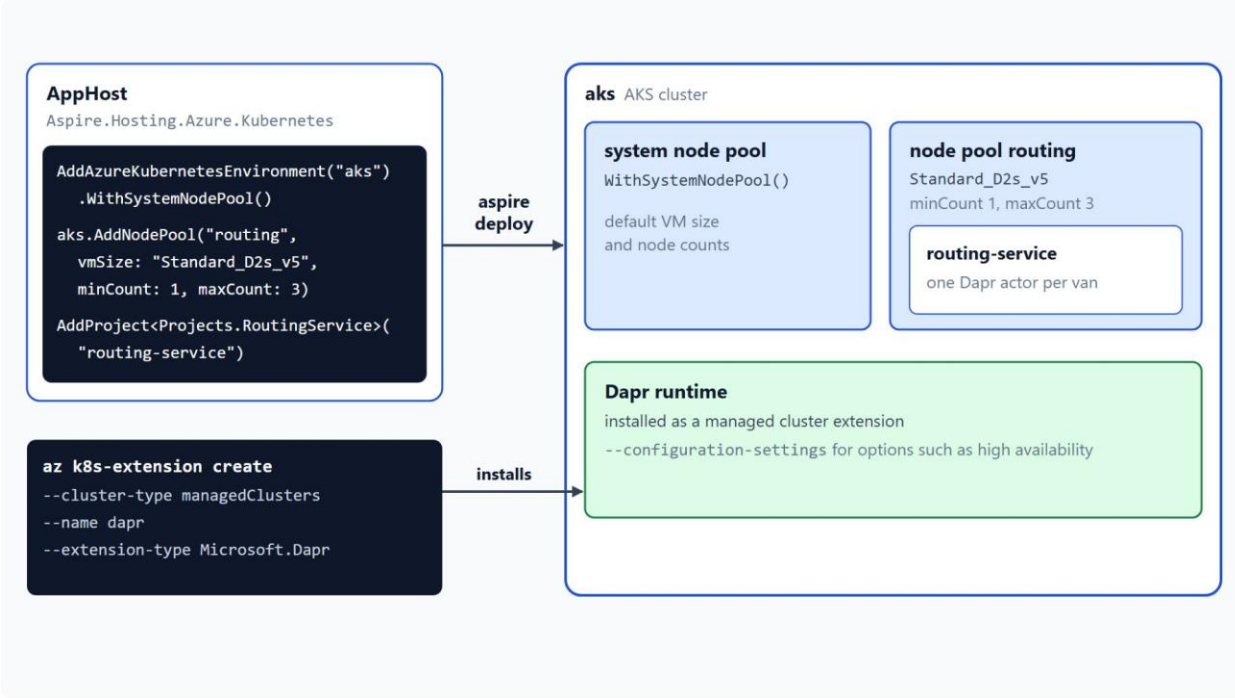
- `aspire publish` writes a **Helm chart** to `aspire-output`; `-o` changes the folder
- A pipeline applies the chart with `helm` or a CI job
- Publishing leaves the cluster unchanged, so you can review the chart first
- The **AKS target** adds Bicep: `aspire deploy` provisions and deploys in one command
- The Aspire CLI reference labels both commands as preview



AKS environment and the Dapr extension

A host where you control the Dapr extension

- `AddAzureKubernetesEnvironment` declares the cluster as a compute environment
- `WithSystemNodePool` and `AddNodePool("routing", ...)` set the node pools
- `WithContainerRegistry`, `WithWorkloadIdentity` and `WithSubnet` fit existing resources
- The **Dapr extension** installs the runtime as a managed cluster extension
- Ingress and Gateway API on AKS are preview in the Aspire documentation



05

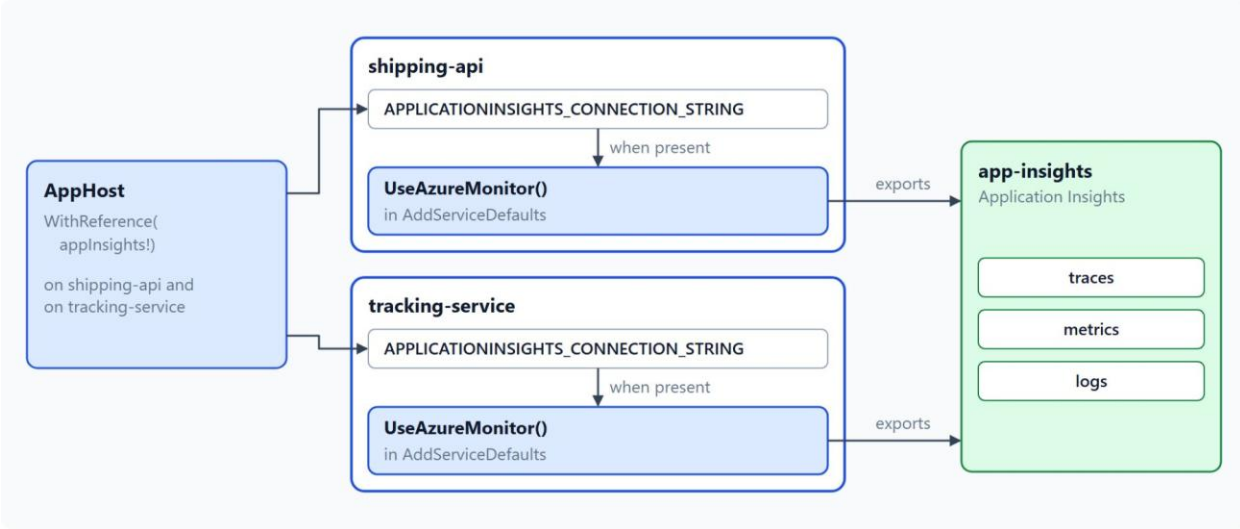
Application Insights: Traces & KQL

The Azure Monitor distro, trace context and KQL

Sending telemetry to Application Insights

The Azure Monitor distro in the shared service defaults

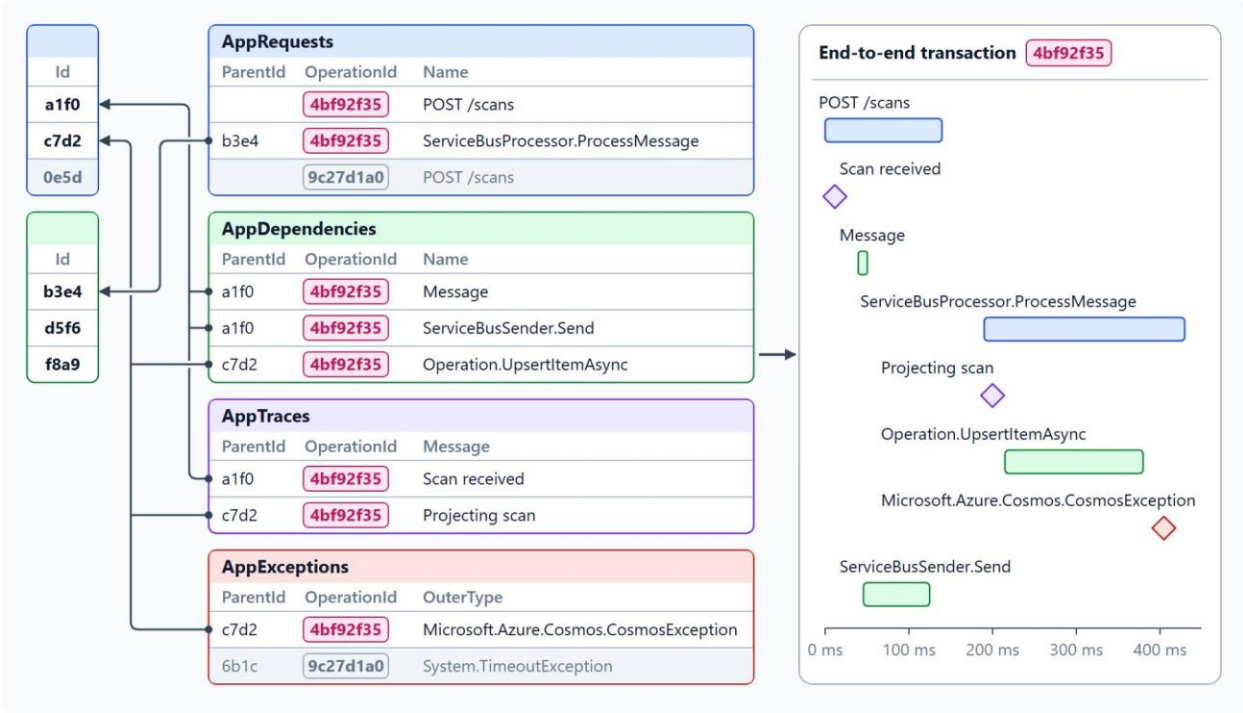
- `UseAzureMonitor` adds traces, metrics and logs to an ASP.NET Core service
- The distro reads `APPLICATIONINSIGHTS_CONNECTION_STRING`
- `AddAzureApplicationInsights` and `WithReference` set it in publish mode
- The **managed OpenTelemetry agent** receives OTLP over gRPC (preview)
- Learn recommends the Microsoft OpenTelemetry Distro for new .NET apps



Trace context and telemetry tables

OperationId and ParentId join the rows of one scan

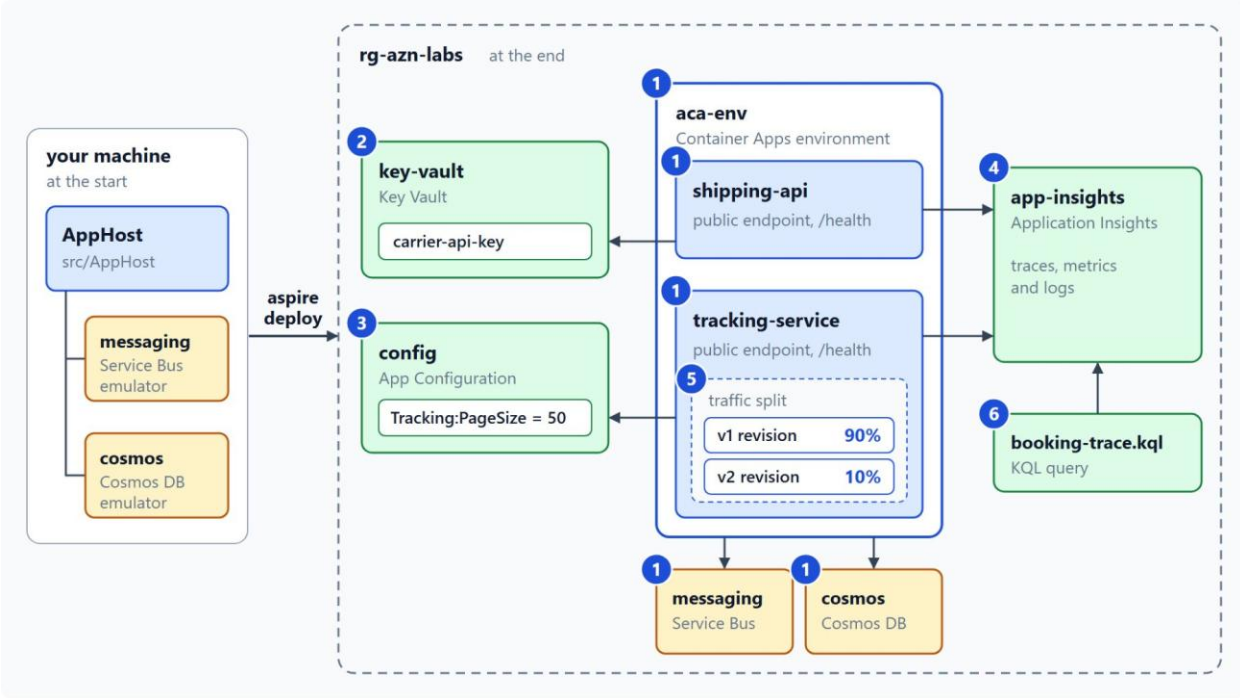
- The Service Bus SDK writes the trace context as `Diagnostic-Id` in `traceparent` format
- Send, receive, process and complete spans link through a `Message` span
- `AppRequests`, `AppDependencies`, `AppTraces` and `AppExceptions`: one table per type
- `OperationId` and `ParentId` join the rows of one transaction
- The end-to-end transaction view draws them as a **Gantt chart**



Shipping Contoso Logistics to Container Apps

What runs in rg-azn-labs at the end of the lab

- Step 1: `aspire deploy` creates `aca-env` with public `shipping-api` and `tracking-service`
- Steps 2 and 3: `carrier-api-key` from **Key Vault**, `Tracking:PageSize` from **App Configuration**
- Step 4: both services export telemetry to **Application Insights**
- Step 5: tracking-service v2 behind a **90/10 traffic split**
- Step 6: `booking-trace.kql` follows one booking across both services



Following one booking with KQL

One query returns every row of the booking

- `POST /shipments` with tracking number `CL-2026-000142` returns 201 Created
- shipping-api writes to `shipments` and sends `ShipmentBooked` to `shipment-events`
- `ShipmentBookedProcessor` in tracking-service writes the booking to `tracking`
- `booking-trace.kql` takes the **OperationId** of the latest shipping-api POST
- It unions `AppRequests`, `AppDependencies` and `AppTraces` in time order



Takeaways

What to carry into reliable messaging with Service Bus

- Declare the environment in the AppHost and let `aspire deploy` build the rest
- Read secrets and settings through a **managed identity**, so no password is stored
- Release behind a **traffic split** and keep the previous revision for rollback
- Give consumers a grace period longer than the host shutdown timeout
- Run Dapr actors on AKS, and join one scan's telemetry by `OperationId`

